

INTRODUCTION TO

Python Programming

A comprehensive course for absolute beginners

Course Objectives

1. Understand

Python syntax, semantics, and the way the interpreter thinks about your code.

2. Build

Programs using variables, operators, control structures, and modular design.

3. Manipulate

Data with lists, tuples, sets, dictionaries, and powerful list comprehensions.

4. Apply OOP

Classes, objects, inheritance, polymorphism, and encapsulation in real designs.

5. Handle errors

Use exceptions, files, and external libraries to write robust programs.

6. Ship a project

Combine everything into one original Python application of your own.

Learning Outcomes

- Write and execute Python programs from a terminal and from a code editor
- Solve computational problems using clear, well-structured Python code
- Create reusable functions and organize them into modules and packages
- Read from and write to files, including CSV and JSON
- Apply object-oriented programming concepts to model real-world problems
- Build small Python applications that connect several concepts together
- Read other people's Python code with confidence — and learn from it

Assessment

Assessment Item	Weight	Notes
Assignments	20%	Take-home problems that extend in-class examples
Practical Tests	20%	Lab-based exercises, completed live in class
Midterm Test	10%	Written test on Modules 1–7
Final Project	20%	An original Python application + report + demo
Final Examination	30%	Written and practical exam covering all modules

How to Use this Deck

- One module per week — each module is one section of the slide deck
- Type every code example into your computer; do not just read it
- Predict the output before you press Run, then check whether you were right
- Complete every practical exercise; the learning sticks at the keyboard, not on the page
- Cross-read the same topic on the W3Schools tutorial when something is unclear
- Ask questions early — a small confusion now becomes a big one in two weeks

MODULE 01

Introduction to Python

What Python is, how to install it, and your very first program

What is Python?

- A high-level, general-purpose programming language created by Guido van Rossum in 1991
- Designed to be readable — code looks almost like English
- Interpreted, so you write and run code without a separate compile step
- Cross-platform: the same script runs on Windows, macOS, and Linux
- Used in web development, data science, automation, scientific computing, and more

Features of Python

Simple syntax

Indentation defines blocks — no curly braces, no semicolons cluttering every line.

Dynamic typing

Variables don't need a type declared; Python infers it from the value.

Cross-platform

Runs on every major operating system without changes to the source code.

Huge standard library

Hundreds of useful modules ship with Python out of the box.

Massive ecosystem

PyPI hosts more than 400,000 third-party packages.

Multi-paradigm

Supports procedural, object-oriented, and functional styles.

Installing Python

- Download from python.org/downloads
- Choose the latest stable 3.x version
- On Windows: tick "Add Python to PATH"
- On macOS/Linux: also available via Homebrew or apt
- Verify in a terminal with `python --version`
- Pick an editor: VS Code or PyCharm

CODE

```
# Verify the install
$ python --version
Python 3.12.2

# Or on macOS/Linux:
$ python3 --version
Python 3.12.2

# Open the interactive REPL:
$ python
>>> print("Hello, Python!")
Hello, Python!
```

Your First Python Program



```
# hello.py — your first Python program

print("Hello, World!")
print("My name is Ada.")
print("I am learning Python.")

# Save the file, then run from a terminal:
#   python hello.py
#
# You should see three lines of output.
# Congratulations — you are now a Python programmer.
```

Type the code yourself rather than copy-paste — typing forces your brain to slow down and notice the details

Syntax Basics

- Indentation marks blocks (use 4 spaces)
- Statements end at the line break, not with ;
- Comments begin with # and run to the end of the line
- Triple-quoted strings can be used as multi-line comments
- Python is case-sensitive: Name and name are different

CODE

```
if 5 > 2:
    print("Five is bigger.")    # indented block
print("Outside the if.")

# This is a single-line comment
x = 10                        # inline comment

"""
A triple-quoted string
spans several lines and is
often used as a block comment.
"""
```

Practical Exercises

- 1 Install Python and confirm that `python --version` reports a 3.x version
- 2 Write a program that prints your name and your favourite quote on separate lines
- 3 Add comments above each `print()` explaining what the line does
- 4 Intentionally indent a `print()` and read the error message in your own words

MODULE 02

Variables and Data Types

How Python remembers things

Variables

- A variable is a named place in memory that stores a value
- Created the moment you assign with =
- No type declaration needed (dynamic typing)
- Python figures out the type from the value on the right
- Use snake_case for variable names
- Pick descriptive names — total_price beats x

CODE

```
age = 25
name = "Ada"
height = 1.65
is_student = True

print(name, "is", age, "years old.")
# Ada is 25 years old.

print(type(age))      # <class 'int'>
print(type(name))     # <class 'str'>
```

Built-in Data Types

Type	Example	Description
int	42	Whole numbers, positive or negative, no decimal
float	3.14	Numbers with a decimal point
str	"hello"	Text — a sequence of characters in quotes
bool	True / False	The two boolean values
NoneType	None	A special value meaning "nothing"
list	[1, 2, 3]	Ordered, mutable collection
tuple	(1, 2, 3)	Ordered, immutable collection
dict	{"a": 1}	Key-value store

Type Conversion (Casting)

- `input()` always returns a string
- Use `int()`, `float()`, `str()` to convert
- Casting may fail (`ValueError`) — validate first
- Conversion never modifies the original; it returns a new value

CODE

```
# Read from the user
year_text = input("Birth year: ")
year = int(year_text)
age = 2026 - year

# float and str
price = float("9.99")
label = "Total: " + str(price)

print(label)           # Total: 9.99

# This raises ValueError:
# int("hello")
```


Naming Rules and Style



```
# ✓ Good – descriptive, snake_case
total_price    = 100
first_name     = "Ada"
MAX_RETRIES    = 3           # constants in UPPER_CASE
is_logged_in   = False

# ⚠ Legal but discouraged
TotalPrice = 100             # CapWords is for class names
firstname  = "Ada"           # missing the underscore
x          = 100             # what does x mean?

# ✗ Illegal
# 1score = 50                # cannot start with a digit
# total-price = 100          # hyphen is not allowed
# class = "A"                # 'class' is a reserved keyword
```

Names cannot start with a digit, contain hyphens, or use any of Python's 35 reserved keywords

Practical Exercises

- 1 Create variables for your full name, age, and height; print them on one line
- 2 Convert a Celsius temperature to Fahrenheit using $F = C * 9/5 + 32$
- 3 Read a price and a quantity from the user; print the total cost
- 4 Try using a Python keyword (such as for or class) as a variable name and read the error
- 5 Read a year of birth and print the age the person will turn this year

MODULE 03

Python Operators

The symbols that combine values into new values

Arithmetic Operators

Operator	Meaning	Example	Result
+	Addition	5 + 2	7
-	Subtraction	5 - 2	3
*	Multiplication	5 * 2	10
/	Division (always float)	5 / 2	2.5
//	Floor division	5 // 2	2
%	Modulus (remainder)	5 % 2	1
**	Exponent (power)	5 ** 2	25

Comparison & Logical

- Comparison operators return True or False
- `==` `!=` `<` `>` `<=` `>=`
- Logical operators: and, or, not (words, not symbols)
- Use parentheses freely — they make intent obvious

CODE

```
age = 18
has_id = True

print(age == 18)           # True
print(age != 21)          # True
print(age >= 18 and has_id) # True
print(age >= 18 or has_id)  # True
print(not has_id)          # False

# Beware: = vs ==
# = is assignment, == is comparison
```

Identity vs Membership

Identity (is, is not)

Identity asks: are these two names the same object in memory?
For everyday programming, prefer == unless you really mean object identity.

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b) # True
print(a is b) # False
```

Membership (in, not in)

Membership asks: does this value appear inside a sequence?

```
fruits = ["apple", "banana"]
print("banana" in fruits)    # True
print("grape" not in fruits) # True

text = "hello world"
print("world" in text)      # True
```

Mini-Project: A Simple Calculator



```
a = float(input("First number: "))
b = float(input("Second number: "))
op = input("Operation (+, -, *, /): ")

if op == "+":
    print("Result:", a + b)
elif op == "-":
    print("Result:", a - b)
elif op == "*":
    print("Result:", a * b)
elif op == "/":
    if b == 0:
        print("Cannot divide by zero.")
    else:
        print("Result:", a / b)
else:
    print("Unknown operation:", op)
```

*Extend the calculator with //, %, and ** for the homework*

Practical Exercises

- 1 Extend the calculator to also support `//` (floor division), `%` (modulus), and `**` (power)
- 2 Read an age and print whether the person is a minor, an adult, or a senior
- 3 Read a number and print whether it is even or odd using the `%` operator
- 4 Predict $(5 + 3) * 2 ** 2 - 4 / 2$ by hand, then run it
- 5 Write a program that asks for two numbers and prints the larger using `max()`

MODULE 04

Python Strings

Working with text — the data of the human world

String Basics

- Single, double, or triple quotes are all valid
- Triple quotes allow strings to span multiple lines
- Concatenate with + and repeat with *
- len(s) returns the number of characters
- Strings are immutable — methods return new strings

CODE

```
name1 = "Ada"
name2 = 'Grace'
poem = """Roses are red,
Violets are blue."""

print(name1 + " " + name2)      # Ada Grace
print("=" * 20)                # 20 equal signs
print(len(name1))              # 3
```

String Slicing

- Each character has an index; the first is 0
- `s[start:stop:step]` is the slicing syntax
- stop is excluded; negative indices count from the end
- Reverse a string with `s[::-1]`
- Slicing produces a new string — original is unchanged

CODE

```
text = "Programming"

print(text[0])      # P
print(text[-1])     # g
print(text[0:6])    # Progra
print(text[7:])     # ming
print(text[:4])     # Prog
print(text[:2])     # Pormig
print(text[::-1])   # gnimmargorP
```

Common String Methods

Method	What it does
<code>.upper()</code> / <code>.lower()</code>	Convert case
<code>.title()</code>	Capitalize each word
<code>.strip()</code>	Remove leading and trailing whitespace
<code>.replace(a, b)</code>	Replace every a with b (returns a new string)
<code>.split(sep)</code>	Break into a list at every separator
<code>.startswith(p)</code> / <code>.endswith(p)</code>	Boolean test for prefix / suffix
<code>.find(t)</code>	Index of first occurrence, or -1
<code>len(s)</code>	Built-in: number of characters

String Formatting

- f-strings are the modern, readable way (Python 3.6+)
- .format() is older but still common
- % style is legacy — avoid in new code
- Format specifiers: :.2f, :>10, :, :%, :b
- Expressions can appear inside the braces

CODE

```
name = "Ada"
age = 36
pi = 3.14159265

# f-string (recommended)
print(f"My name is {name} and I am {age}.")
print(f"Pi: {pi:.2f}")           # 3.14
print(f"Right-aligned: {age:>5}")
print(f"Computed: {2 + 3}")

# .format() — same idea
print("Hello, {}".format(name))
```

Mini-Project: Username Formatter



```
# Build a tidy username from a first and last name.

first = input("First name: ").strip()
last  = input("Last name: ").strip()

# initial of first + lastname, all lowercase
username = (first[0] + last).lower()

# Greet with proper title case
greeting = (
    f"Hi {first.title()} {last.title()}, "
    f"your username is {username}."
)
print(greeting)

# Example:
# First name: ada
# Last name: Lovelace
# Hi Ada Lovelace, your username is alovelace.
```

Try entering names with uppercase letters and extra spaces — the `.strip()` and `.lower()` handle them gracefully

Practical Exercises

- 1 Read a sentence; print its length, uppercase form, and underscored form
- 2 Read a full name and print just the initials separated by dots
- 3 Detect whether a word is a palindrome (reads the same forwards and backwards)
- 4 Read an email address and print the part before and after the @
- 5 Print a 20-wide hollow rectangle of asterisks using only string operations

MODULE 05

Python Collections

Lists, tuples, sets, dictionaries — the workhorses of Python

The Four Built-in Collections

List `[a, b, c]`

Ordered, mutable. The default container — change its contents freely.

Tuple `(a, b, c)`

Ordered, immutable. For fixed groups of related values like coordinates.

Set `{a, b, c}`

Unordered, unique values. Membership tests and de-duplication are fast.

Dict `{k: v, k: v}`

Key–value pairs. The workhorse for record-like data.

Working with Lists

- Created with square brackets: `fruits = []`
- Access by index, slice like strings
- `.append()` adds to the end
- `.insert(i, x)` places `x` at index `i`
- `.remove(x)` deletes the first match
- `.sort()` sorts in place; `sorted()` returns a new list

CODE

```
fruits = ["apple", "banana", "cherry"]

fruits.append("date")
fruits.insert(1, "blueberry")
print(fruits[0])           # apple
print(fruits[-1])          # date
print(fruits[1:3])

fruits.sort()
print(fruits)
fruits.remove("banana")
print(fruits)
```

List Comprehensions



```
# Build new lists in one expressive line.
```

```
squares = [n * n for n in range(1, 11)]  
print(squares)  
# [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

```
# With a filter  
nums = [1, 2, 3, 4, 5, 6, 7, 8]  
evens = [n for n in nums if n % 2 == 0]  
print(evens)          # [2, 4, 6, 8]
```

```
# Transformation  
words = ["python", "is", "fun"]  
shouted = [w.upper() for w in words]  
print(shouted)        # ['PYTHON', 'IS', 'FUN']
```

```
# Nested comprehension (2D grid)  
grid = [[r * c for c in range(1, 4)] for r in range(1, 4)]  
print(grid)
```

Comprehensions are one of Python's most distinctive and beautiful features

Tuples & Sets

- Tuples are immutable; once created, contents cannot change
- Single-element tuple needs the trailing comma: (42,)
- Sets store unique values — duplicates vanish automatically
- Sets support union (|), intersection (&), difference (-)
- Frozensets are immutable sets (and can be dictionary keys)

CODE

```
# Tuple – fixed group
point = (3, 4)
x, y = point          # unpack

# Set – unique values
tags = {"py", "code", "py", "fun"}
print(tags)           # duplicates removed

a = {1, 2, 3}
b = {3, 4, 5}
print(a | b)          # union {1,2,3,4,5}
print(a & b)           # intersection {3}
print(a - b)          # difference {1, 2}
```

Dictionaries



```
# A dictionary stores key-value pairs.
student = {
    "name": "Ada Lovelace",
    "age": 36,
    "grades": [90, 85, 92],
}

# Read and write by key
print(student["name"])          # Ada Lovelace
student["email"] = "ada@x.com"  # add a new field

# .get() never raises a KeyError
print(student.get("phone", "n/a"))

# Iterate over key/value pairs
for key, value in student.items():
    print(f"{key}: {value}")

# Nested dictionary — a small in-memory database
students = {
    "S001": {"name": "Ada", "grade": "A"},
    "S002": {"name": "Grace", "grade": "B"},
}
print(students["S001"]["name"])
```

Think of dictionaries as small in-memory databases — they are everywhere in Python

Practical Exercises

- 1 Make a list of your five favourite movies and print it sorted alphabetically
- 2 Read 5 numbers; print the largest, smallest, and average using built-ins
- 3 Use a set to remove duplicates from a list of words
- 4 Build a dictionary of country → capital and look up countries from input
- 5 Use a list comprehension to produce all even squares from 1 to 100

MODULE 06

Decision Making

Programs that choose what to do next